BUILD REPORT · END TO END

SRE Copilot

Evidence-backed, read-only infrastructure diagnosis — from the first foundation commit through a live Kubernetes console and self-hosted accounts.

REPOSITORY
github.com/Swastik258/sre-copilot

BRANCH · COMMIT
master · 3fec12f

DATE
19 October 2026

SCOPE
Milestones 1, 2, UI redesign, 2.5 (accounts)

9	164	82	31
COMMITS	TRACKED FILES	TESTS PASSING	ROUTES BUILT

Everything in this report describes code that is committed and pushed to **master**. Where a capability is simulated, unverified, or not built, it is labelled **Demo**, **Preview**, or **Planned**. No customer logos, testimonials or production usage are claimed.

CONTENTS

What this report covers

A complete inventory of what was built, how it fits together, and what is honestly still missing.

03	Executive summary	What works, what does not
04	Feature inventory — platform & Kubernetes	18 capabilities
05	Feature inventory — accounts, UI & planned work	14 capabilities
06	Product walkthrough — signup & dashboard	Screenshots
07	Product walkthrough — console & marketing	Screenshots
08	Architecture & request lifecycle	How the pieces connect
09	Authentication deep dive	Hashing, sessions, enforcement
10	API reference	15 endpoints
11	Data model & migrations	8 tables, 3 revisions
12	Frontend route map	29 pages
13	File inventory — added files	164 total · 15 new
14	File inventory — modified files	21 updated
15	Testing & verification	Commands and results
16	Security & limitations	Controls and gaps
17	Run, deploy, known issues, next	Practical reference

How to read the status labels

Implemented	Real code, exercised by automated tests or a reproducible end-to-end check.
Demo verified	Works end to end against a labelled fixture snapshot. Not verified against a live third-party system.
Preview only	A designed UI surface with example content. It does not perform the action it describes.
Planned	Not built. Named so the gap is explicit rather than implied.

> **Two things to keep in mind.** First, the API is not protected by default: local accounts exist, but enforcement is behind `AUTH_REQUIRED`, which is **off** so the demo and test suite run without credentials. Second, AI reasoning and the GitHub, Prometheus and Terraform adapters are **not implemented**; the console diagnoses Kubernetes only.

01 · EXECUTIVE SUMMARY

A tested foundation, one real diagnosis engine, and real accounts

The project is a read-only SRE copilot. It collects operational evidence, applies deterministic rules, and returns findings that separate observed facts from hypotheses and next steps. This engagement took it from a scaffold to a working Kubernetes diagnosis console with self-hosted signup and login.

What works today

- **Kubernetes diagnosis.** Pod, event, workload and log collection through a structurally read-only client, with rule-based findings and cited evidence.
- **Five failure classes.** Crash loops, OOM kills, image-pull failures, unschedulable pods, and probe/non-zero-exit failures.
- **Evidence model.** Every finding points at evidence with source, provenance, redaction and a demo flag.
- **Local accounts.** Signup, login, logout and session lookup, with scrypt-hashed passwords and server-side sessions.
- **Gated dashboard.** Authenticated users land on `/dashboard`; the middleware redirects anonymous visitors to `/login`.
- **Operational plumbing.** Health checks, request IDs, structured logs, uniform errors, redaction, body limits and optional rate limiting.

What does not work yet

- **AI reasoning.** No model is called. Diagnoses are deterministic rules, not LLM output.
- **Other adapters.** GitHub Actions, Prometheus and Terraform are status entries only.
- **Investigations.** The database schema exists, but there is no API or UI to save or replay one.
- **Authorization.** No roles, permissions or tenant isolation. Any authenticated user is equivalent.
- **Account lifecycle.** No email verification, password reset, MFA or lockout.
- **Live cluster proof.** Verified against demo fixtures only; this environment has no Docker daemon or kubeconfig.

> **Bottom line.** The Kubernetes path is real and tested end to end. The account path is real and tested end to end. Everything else in the product surface is either a clearly labelled preview or explicitly planned. The deployment is not production-hardened: authentication enforcement is off by default and there is no authorization model.

Delivery at a glance

METRIC	VALUE	EVIDENCE
Backend test suite	**82 passing**	`pytest`, 1 third-party deprecation warning
Backend lint	**Clean**	`ruff check .`
Frontend lint	**Clean**	`npm run lint`
Production build	**31 routes**	`npm run build`
Browser E2E	**8 flows**	Playwright against the built app
Source lines	**4,785 Python · 6,182 TS/TSX**	`wc -l` over tracked files

02 · FEATURE INVENTORY

Platform, operations and Kubernetes

Each row is a capability that exists in the repository, with its real status.

AREA	CAPABILITY	STATUS
API platform	Versioned REST API under `/api/v1` with OpenAPI docs	Implemented
	Uniform JSON error envelope, including safe validation details	Implemented
	Request IDs propagated into structured JSON logs	Implemented
	Request-size limit and optional in-memory rate limiting	Implemented
Safety & hygiene	Server-side redaction of credentials, tokens and private keys	Implemented
	CORS with an explicit origin allow-list and credentials support	Implemented
	Demo data labelled end to end with a `demo` flag	Implemented
	Audit events written for security-relevant actions	Implemented
Health & status	Health, liveness and readiness endpoints with a real database check	Implemented
	Integration status endpoint that never returns secrets	Implemented
Kubernetes	Structurally read-only client interface with no mutating methods	Implemented
	Read-only check enforced by a dedicated test	Implemented
	Real client using the official Kubernetes client, lazily imported	Implemented
	Fixture client serving a labelled demo cluster snapshot	Demo verified
	Normalised pods, container statuses, events, workloads, namespaces, bounded logs	Implemented
	Deterministic rules for crash loops, OOM, image pull, pending/probe, exits, log leads	Implemented
	Live-cluster verification	Not verified

How a diagnosis is produced

- The console selects a pod from the read-only API.
- The service collects pod state, container statuses, events and bounded logs.
- Deterministic evaluators emit findings and evidence objects.
- Confidence is derived from the findings; risk is derived from severity.
- Next steps are read-only commands. Nothing mutates the cluster.

What is deliberately absent

- No `create`, `patch` or `delete` in the client interface.
- No shell execution or tool-calling into the cluster.
- No remediation is applied automatically; the UI presents it behind an explicit approval step.
- No claim that a live cluster has been tested in this environment.

02 · FEATURE INVENTORY (CONTINUED)

Accounts, interface and planned work

AREA	CAPABILITY	STATUS
Accounts (M2.5)	Signup, login, logout and current-user endpoints	Implemented
	scrypt password hashing with a PBKDF2 fallback, per-user salt, constant-time compare	Implemented
	Server-side sessions; the token is stored only as a SHA-256 hash	Implemented
	Session delivered as an `HttpOnly`, `SameSite=Lax` cookie	Implemented
	Gated `/dashboard` with live integration status and quick links	Implemented
	Auth-aware navigation, including the mobile menu	Implemented
	Optional API-wide enforcement through `AUTH_REQUIRED`	Implemented off by default
Interface	Dark, responsive product site (29 pages) with motion and reduced-motion support	Implemented
	Kubernetes console at `/kubernetes` with namespace filter, pod list, diagnosis and evidence	Implemented
	Seven additional product surfaces (assistant, logs, CI/CD, Terraform, infra code, audit, settings)	Preview only
	Marketing pages: product, solutions, pricing, customers, resources, docs, security, legal	Implemented static content
Planned	AI provider integration and model-backed reasoning	Planned
	GitHub Actions, Prometheus and Terraform adapters	Planned
	Investigation persistence, replay and sharing	Planned schema only
	Roles, permissions and tenant isolation	Planned
	Email verification, password reset, MFA and account lockout	Planned
	Billing, usage metering and production deployment hardening	Planned

Preview surfaces are honest about it. Pages such as `/logs` and `/cicd` render example content and disabled controls. They are not wired to an adapter, and the UI does not claim otherwise. The testimonials page states plainly that no quotes will appear until partners agree to be named.

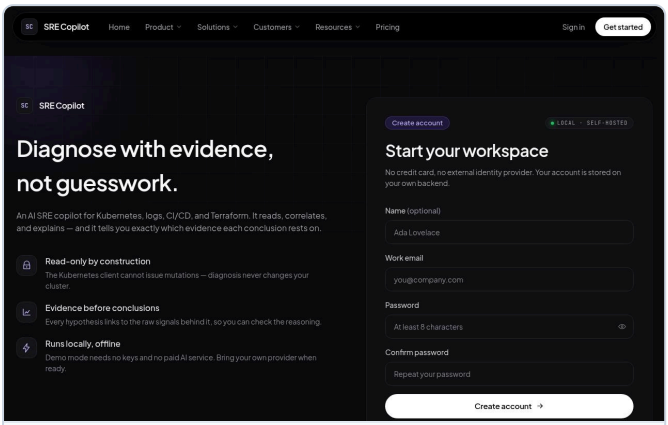
Principles applied

- Evidence before conclusions; facts separated from hypotheses.
- Read-only access by default; no mutating cluster methods.
- Never fabricate logs, metrics, events or results.
- Human approval required before any infrastructure-changing action.
- Work locally with no paid services or third-party credentials.
- Clearly distinguish live integrations from simulated demonstrations.

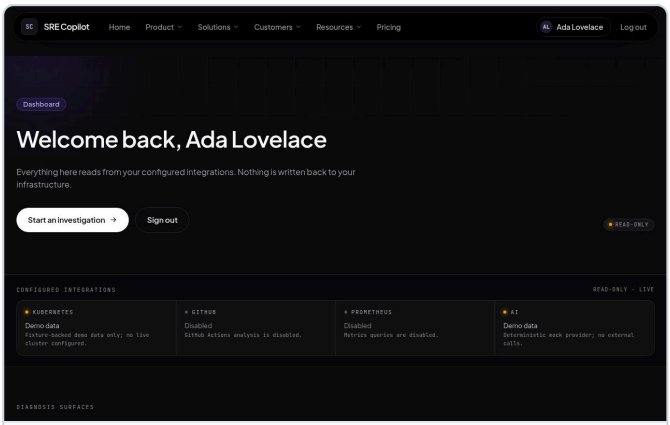
03 · PRODUCT WALKTHROUGH

Signup and the authenticated dashboard

Captured from the production build running locally against the real API and a local SQLite database.



/sign-up — split layout: brand panel and form. Inline validation, password reveal, and a link to sign in.



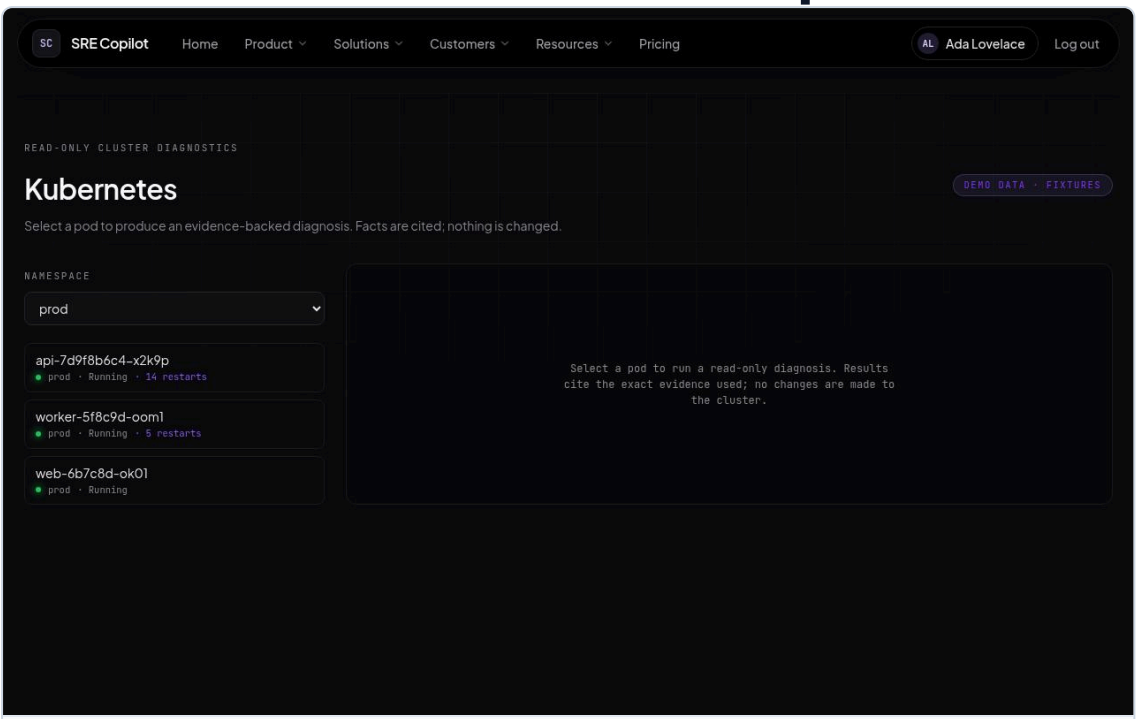
/dashboard — gated. Greeting, live integration strip, six diagnosis surfaces, account card and a “Planned · M3” investigations panel.

Verified flow

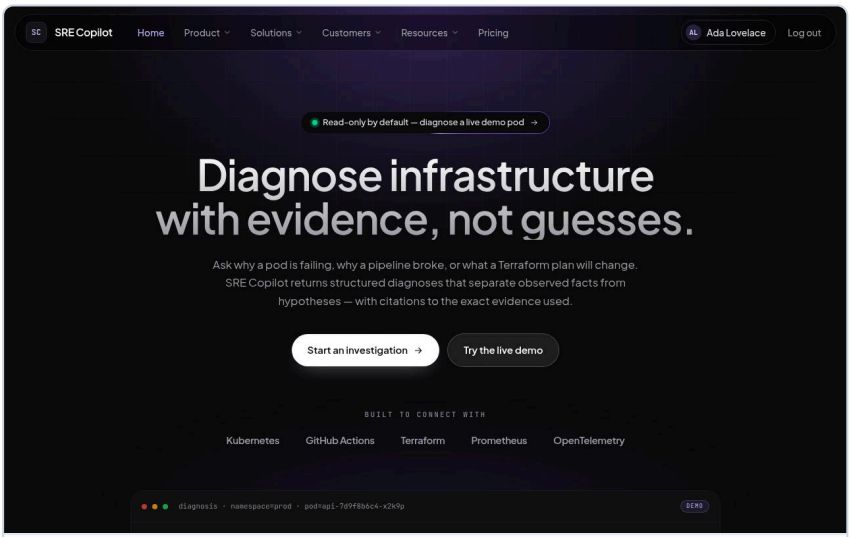
1. **Logged out.** The navigation shows **Sign in** and **Get started**; no console link is exposed as the primary action.
2. **Signup.** Name, email, password and confirmation are validated in the browser, then posted to the API.
3. **Session created.** The API returns `201` with the user only; the token is set in an `HttpOnly` cookie and never appears in the response body.
4. **Dashboard.** The client mirrors identity through `/auth/me` and the router lands on `/dashboard`.
5. **Reload.** The session persists; the navigation shows the user’s initials and a log-out control.
6. **Logout.** The session row is revoked, the cookie is cleared, and the app returns to `/login`.
7. **Guard.** Visiting `/dashboard` while logged out redirects to `/login`.

> The account shown is a throwaway demo user created during verification (`report.…@example.com`). It is not a real customer or a seeded production account.

The Kubernetes console and the public site



/kubernetes — read-only console. Namespace filter, pod list, evidence-backed diagnosis and a “demo data · fixtures” badge when the fixture client is active.



/ — dark marketing home. Product narrative, live integration status, and CTAs that now lead to signup rather than a bare console.

What the console does

- Lists clusters, namespaces, workloads and pods from the read-only API; the fixture client is used when demo mode is enabled.
- Runs a diagnosis for a selected pod and renders findings with confidence, risk and next steps.
- Shows the underlying evidence with source, provenance, severity and a demo badge where applicable.
- Presents remediation behind an explicit approval step; the client interface cannot mutate the cluster.

> **Demo boundary.** These screenshots were taken with `KUBERNETES_ENABLED=true` and `KUBERNETES_DEMO_MODE=true`. The fixture cluster is a labelled snapshot, not a live environment.

04 · ARCHITECTURE

How the pieces connect

A single Next.js client talks to a versioned FastAPI service. Diagnosis logic lives in services; external systems sit behind adapters.



Cross-cutting middleware — request context and IDs → CORS → body-size limit → rate limit → structured logging → uniform exception handlers.

Persistence — SQLAlchemy 2.x over SQLite locally or Postgres in Compose; Alembic owns production schema changes.

Evidence — every finding cites evidence objects carrying source resource, provenance, severity, redaction and a demo flag.

Request lifecycle — an authenticated diagnosis

1. The browser calls the API with the session cookie attached.
2. Middleware assigns a request ID, applies CORS and limits, and logs the request.
3. The route dependency resolves the session (`enforce_auth` is a no-op unless `AUTH_REQUIRED=true`).
4. The Kubernetes service collects normalised pod, event and log data through the read-only adapter.
5. Deterministic evaluators produce findings plus evidence; provenance and redaction are applied.
6. The response is validated against a typed schema and returned; failures use the JSON error envelope.

Why this shape

Adapters isolate every external system, so a read-only Kubernetes client can be swapped for a fixture without touching diagnosis rules. Evidence is a first-class model rather than a string in a response, which is what makes “explain why” and later audit/replay possible. Auth is a service plus a dependency, so enforcement can be turned on without rewriting routes.

05 · AUTHENTICATION

Self-hosted accounts, end to end

No third-party identity provider is required. Accounts and sessions live in the application database.

Signup and login flow

1. **Submit.** The form posts email and password to `/api/v1/auth/signup` or `/login`.
2. **Validate.** Email is normalised and checked; passwords require at least eight characters.
3. **Hash or verify.** Signup derives a scrypt hash with a fresh 16-byte salt. Login uses a constant-time comparison.
4. **Create a session.** A 256-bit opaque token is generated; only its SHA-256 hash is stored, with a seven-day expiry.
5. **Set the cookie.** `sre_session` is `HttpOnly`, `SameSite=Lax`, scoped to `/`, and `Secure` outside local development.
6. **Return identity only.** The response contains the user record; the raw token is never in a response body.
7. **Resolve later requests.** The dependency accepts the cookie (or a Bearer token for tooling) and looks up the unrevoked, unexpired session hash.

Controls and parameters

CONTROL	IMPLEMENTATION
Password hashing	stdlib `scrypt`, $N=2^{14}$, r=8, p=1, 32-byte key, 16-byte per-user salt; PBKDF2-HMAC-SHA256 at 200,000 iterations as a fallback
Comparison	Constant-time (`hmac.compare_digest`)
Session token	`secrets.token_urlsafe(32)` — 256 bits of entropy
At-rest token	SHA-256 hex digest only; a database leak does not yield usable sessions
Cookie attributes	`HttpOnly`, `SameSite=Lax`, `Path=/`, `Secure` when not local
Session lifetime	168 hours by default; revocable immediately on logout
Unique accounts	Unique index on `users.email`; duplicate signup returns a conflict
Audit	Signup, login and logout events recorded

> **Enforcement is opt-in.** `AUTH_REQUIRED=false` by default. When true, `/api/v1/integrations` and `/api/v1/kubernetes/*` require a valid session; health and the auth endpoints stay public. The frontend guard checks only that a cookie is present — real verification happens on every API call.

Clerk remains an optional alternative for the navigation when its keys are present at build time. Without them, the app uses these local pages and requires no third-party credentials.

06 · API REFERENCE

Fifteen endpoints under `/api/v1`

Generated from the routers in `backend/app/api/v1/`.

METHOD	PATH	ACCESS	PURPOSE
GET	`/health`	Public	Health with a real database check
GET	`/health/live`	Public	Process liveness
GET	`/health/ready`	Public	Readiness, including the database
GET	`/integrations`	Gated*	Configuration status; never returns secrets
GET	`/kubernetes/clusters`	Gated*	Available clusters and demo flag
GET	`/kubernetes/namespaces`	Gated*	Namespace list
GET	`/kubernetes/workloads`	Gated*	Deployments, stateful sets, daemon sets
GET	`/kubernetes/pods`	Gated*	Normalised pod list, optionally filtered
GET	`/kubernetes/pods/{ns}/{name}`	Gated*	Pod detail with container statuses
GET	`/kubernetes/pods/{ns}/{name}/logs`	Gated*	Bounded, redacted log lines
POST	`/kubernetes/pods/{ns}/{name}/diagnose`	Gated*	Findings, evidence, confidence, next steps
POST	`/auth/signup`	Public	Create an account; sets the session cookie
POST	`/auth/login`	Public	Authenticate; sets the session cookie
POST	`/auth/logout`	Public	Revoke the session and clear the cookie
GET	`/auth/me`	Cookie / Bearer	Current user identity

* Gated only when `AUTH_REQUIRED=true`. Otherwise the read-only demo routes are open.

Error envelope

Every handled failure returns one shape, so the client never parses ad-hoc errors:

```
{ "error": { "code": "validation_error", "message": "...",
             "details": [...], "request_id": "..." } }
```

HTTP	CODE	MEANING
400	`bad_request`	Malformed request
401	`unauthorized`	Missing or invalid credentials (enforced only with `AUTH_REQUIRED`)
403	`permission_denied`	Not allowed
404	`not_found`	Resource missing
409	`conflict`	State conflict, such as an existing email
413	`payload_too_large`	Body exceeds the configured limit
422	`validation_error`	Schema validation failed
500	`internal_error`	Unexpected failure, logged with a request ID

07 · DATA MODEL

Eight tables and three migrations

SQLAlchemy 2.x models, created by `create_all` for local SQLite and by Alembic for everything else.

TABLE	KEY COLUMNS	ROLE
`users`	id, external_id, email (unique), display_name, password_hash, is_active, created_at, updated_at	Local and externally authenticated accounts
`sessions`	id, user_id → users, token_hash (unique), user_agent, created_at, expires_at, revoked_at	Server-side login sessions
`conversations`	id, user_id, title, timestamps	Assistant threads (schema only)
`messages`	id, conversation_id, role, content, timestamps	Thread messages (schema only)
`integrations`	id, kind, name, status, configuration metadata	Integration registry and status
`investigations`	id, title, status, surface, timestamps	Investigation records (no API yet)
`evidence`	id, investigation_id, source_resource, provenance, payload, severity, demo flag	Cited facts behind findings
`audit_events`	id, user_id, action, target, metadata, created_at	Security-relevant activity

Migration history

REVISION	PURPOSE	NOTES
`0001_initial`	Initial schema	Base tables
`0002_core_tables`	Core domain	Evidence, integrations, investigations, conversations, audit
`0003_auth`	Local authentication	Adds `users.password_hash`, a unique email index and `sessions`

> **Local schema upgrades.** `create_all()` never alters an existing table, so a database created before the auth milestone would fail on signup. The backend now applies additive, idempotent upgrades to local SQLite at startup (`app/db/local_schema.py`) and logs exactly what changed. Postgres and shared environments are never altered this way — they use Alembic.

Data-handling rules

- Secrets are read on the backend only and never embedded in the frontend bundle.
- Logs and evidence pass through redaction before storage or model use.
- Evidence keeps its provenance and a demo flag so simulated data is never mistaken for live data.
- Sessions store only a token hash; passwords store only a salted hash.

08 · FRONTEND MAP

Twenty-nine pages, grouped by what they actually do

The site is intentionally larger than the working product. Every non-working surface is a preview, not a claim.

Working product

ROUTE	TYPE	BEHAVIOUR
`/kubernetes`	Live	Namespace filter, pod list, evidence-backed diagnosis
`/dashboard`	Live	Authenticated landing page; gated by middleware
`/login`	Live	Email/password login with error handling
`/sign-up`	Live	Account creation with confirmation and validation

Preview surfaces

ROUTE	TYPE	BEHAVIOUR
`/assistant`	Preview	Example diagnosis conversation; no model is called
`/logs`	Preview	Example log analysis; no log provider is connected
`/cicd`	Preview	Example pipeline failure view; no GitHub adapter
`/terraform`	Preview	Example plan output; no Terraform integration
`/infra-code`	Preview	Example manifest review; static content
`/audit`	Preview	Example audit timeline; database events are real but not surfaced here
`/settings`	Preview	Example workspace settings; not persisted

Marketing and documentation

Static content, built as real pages with real navigation:

`/` · `/product` · `/solutions` · `/solutions/sre` · `/solutions/devops` · `/solutions/platform` · `/pricing` · `/customers` · `/customers/case-studies` · `/customers/testimonials` · `/resources` · `/blog` · `/changelog` · `/support` · `/docs` · `/security` · `/privacy` · `/terms`

Design system

- Dark, high-contrast visual language with one accent colour for emphasis.
- Status colours are reserved for health and severity.
- Responsive from 375 px; navigation collapses to a mobile menu.
- Motion is progressive and respects `prefers-reduced-motion`.
- Shared primitives: buttons, cards, badges, section shells and icon set.
- Auth pages reuse the same shell rather than a separate brand.

09 · FILE INVENTORY

164 tracked files · 36 changed in the accounts milestone

AREA	FILES	CONTENTS
Backend	**79**	App modules, adapters, services, schemas, security, migrations, tests, Dockerfile
Frontend	**66**	App Router pages, components, typed API client, auth context, proxy, config
Documentation	**11**	Architecture, security, API, testing, roadmap, setup guides, review notes
Deploy & root	**8**	Compose file, README, CONTRIBUTING, BUILD_STATUS, SECURITY, .env.example, CI, .gitignore

The accounts milestone touched 36 files: **15 added** and **21 modified**.

+	FILE	PURPOSE
+	`backend/alembic/versions/0003_auth.py`	Password column, unique email index, sessions table
+	`backend/app/api/v1/auth.py`	Signup, login, logout, me; cookie handling
+	`backend/app/db/local_schema.py`	Additive startup upgrades for legacy local SQLite
+	`backend/app/schemas/auth.py`	Email/password validation and auth response models
+	`backend/app/security/passwords.py`	scrypt hashing with PBKDF2 fallback
+	`backend/app/security/tokens.py`	256-bit token generation and SHA-256 hashing
+	`backend/app/services/auth.py`	Account and session service logic
+	`backend/tests/integration/test_auth.py`	12 API tests for the auth lifecycle
+	`backend/tests/unit/test_local_schema.py`	3 tests for legacy database upgrades
+	`frontend/app/dashboard/page.tsx`	Gated post-login dashboard
+	`frontend/app/login/page.tsx`	Login route
+	`frontend/app/sign-up/page.tsx`	Signup route
+	`frontend/components/AuthForm.tsx`	Shared form: validation, errors, password reveal
+	`frontend/components/AuthShell.tsx`	Two-column auth page chrome
+	`frontend/lib/auth-client.tsx`	`AuthProvider` and `useAuth`

> **The other 128 files.** They predate this milestone: the M1 foundation (configuration, logging, errors, redaction, health, evidence, audit, CI, Docker) and M2 (the read-only Kubernetes adapter, rules, schemas, console and fixture data), plus the dark design-system pass.

09 · FILE INVENTORY (CONTINUED)

Twenty-one modified files

Existing files updated to wire authentication through the API, the interface and the documentation.

~	FILE	PURPOSE
~	`.env.example`	Auth settings; credentials enabled with an explicit origin list
~	`BUILD_STATUS.md`	Updated status, test count, auth notes
~	`SECURITY.md`	Corrected the “no authentication” statement
~	`backend/app/api/deps.py`	`get_current_user`, `require_user`, `enforce_auth`
~	`backend/app/api/v1/router.py`	Mount auth; gate data routes
~	`backend/app/core/config.py`	Auth flag, cookie name, session TTL
~	`backend/app/core/errors.py`	Safe serialisation of validation details
~	`backend/app/db/models.py`	Password hash, session model, indexes
~	`backend/app/main.py`	Calls the local schema upgrade
~	`backend/tests/integration/test_migrations.py`	Expects the sessions table
~	`docs/API.md`	Corrected 401 semantics
~	`docs/ARCHITECTURE.md`	Local auth and client notes
~	`docs/LOCAL_DEVELOPMENT.md`	Auth setup and legacy database guidance
~	`docs/ROADMAP.md`	Milestone 2.5 and remaining gaps
~	`docs/SECURITY.md`	Implemented controls and missing ones
~	`frontend/app/docs/page.tsx`	In-app authentication documentation
~	`frontend/app/layout.tsx`	Wraps the app in the auth provider
~	`frontend/app/security/page.tsx`	Honest implemented/missing lists
~	`frontend/components/TopNav.tsx`	Auth-aware navigation and mobile actions
~	`frontend/lib/api.ts`	Auth methods and credential-carrying requests
~	`frontend/proxy.ts`	Redirects anonymous `/dashboard` visits

Where to review it. Start with `app/services/auth.py` and `app/security/passwords.py`, then the migration `0003_auth.py`. The twelve tests in `tests/integration/test_auth.py` cover the full lifecycle, cookie flags, hashing at rest, bearer-token access and the `AUTH_REQUIRED` gate.

10 · VERIFICATION

What was executed, and what it proved

CHECK	COMMAND	RESULT
Backend tests	`pytest`	**82 passed**, 1 warning
Backend lint	`ruff check .`	**Clean**
Frontend lint	`npm run lint`	**Clean**
Production build	`npm run build`	**31 routes**, proxy active
Browser E2E	Playwright, real Chromium	**8 flows passed**
API checks	HTTP with a cookie jar	201 → 200 → 401 → 204
Legacy DB repair	Stale SQLite fixture	500 reproduced → 201 after fix

Test coverage by area

AREA	FILES
Configuration & plumbing	`test_config, test_health, test_errors, test_ratelimit, test_redaction`
Evidence & audit	`test_evidence, test_audit, test_db_persistence`
Integrations	`test_api_integrations`
Kubernetes	`test_kubernetes_api, test_kubernetes_diagnosis, test_kubernetes_fixture_client, test_kubernetes_readonly`
Accounts	`test_auth, test_migrations`
Local schema upgrades	`test_local_schema`

The 82 figure is collected tests; some cases are parametrised, so it exceeds the raw count of test functions.

End-to-end flows verified

1. Logged-out navigation shows **Sign in** and **Get started**.
2. Signup lands on `/dashboard` with the correct greeting.
3. Reload keeps the session; the signed-in chip is present.
4. Logout returns to `/login` and clears the session.
5. Visiting `/dashboard` logged out redirects to `/login`.
6. A wrong password shows “Invalid email or password.”
7. The correct password returns to `/dashboard`.
8. At 375 px, signup and the authenticated dashboard remain usable.

Bug found and fixed during verification

A pre-auth SQLite database produced a 500 on every signup: `no such column: users.password_hash`. The cause was `create_all()` creating missing tables but never altering an existing one. The failure was reproduced on a deliberately stale database, the additive upgrade was added, and the same database then returned `201` for signup, `200` for login, and `200` for `/auth/me` with an `HttpOnly` cookie.

> **What “passed” means here.** Tests and E2E ran locally against SQLite and the fixture Kubernetes client. They do not prove behaviour against a live cluster, Postgres, or a production deployment.

11 · SECURITY

Controls that exist, and controls that do not

Implemented

- ✓ Read-only Kubernetes interface with no mutating methods, enforced by a test.
- ✓ Server-side redaction of credentials, tokens and private keys before storage or model use.
- ✓ scrypt password hashing with per-user salts and constant-time comparison.
- ✓ Opaque session tokens stored only as SHA-256 hashes.
- ✓ Session cookie is `HttpOnly`, `SameSite=Lax`, and `Secure` outside local development.
- ✓ CORS credentials enabled only with an explicit origin allow-list.
- ✓ Request-size limits and optional rate limiting.
- ✓ Structured logs with request IDs; secrets are not logged.
- ✓ Uniform error envelope that does not leak stack traces to clients.
- ✓ API secrets are read on the backend only and never bundled into the frontend.

Not implemented — do not treat this as production auth

- ✗ **No authorization model.** No roles, permissions, workspace membership or tenant isolation.
- ✗ **Enforcement is off by default.** `AUTH_REQUIRED=false` leaves the read-only API open.
- ✗ **No account lifecycle.** No email verification, password reset, MFA, or lockout after repeated failures.
- ✗ **No auth-specific rate limiting.** The general limiter is off by default, so credential stuffing is not throttled.
- ✗ **Frontend guard is presence-only.** `proxy.ts` checks that a cookie exists; the API is the real authority.
- ✗ **No CSRF token.** `SameSite=Lax` mitigates cross-site POSTs but is not a complete CSRF strategy.
- ✗ **No TLS in the app.** Secure cookies depend on termination in front of the service.
- ✗ **No secret storage or rotation** for connected integrations.
- ✗ **No egress controls** for hosted AI providers beyond privacy mode (and no provider is wired yet).

> **Deployment guidance.** Run behind TLS, set `AUTH_REQUIRED=true`, keep the CORS origin list explicit (never `*` with credentials), enable rate limiting, and do not expose the API publicly until authorization and account lifecycle exist.

Reporting

Security issues should be reported privately to the maintainers. Do not include credentials or other secrets in an issue report.

Run it, deploy it, and what comes next

Local — no keys, no cluster, no paid services

```
# backend
cd backend
python -m venv .venv && . .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -e ".[dev]"
uvicorn app.main:app --reload --port 8000

# frontend (new terminal)
cd frontend
npm install
NEXT_PUBLIC_API_URL=http://localhost:8000 npm run dev
```

Backend docs at `http://localhost:8000/docs`; app at `http://localhost:3000`. The local SQLite schema is created automatically. To exercise Kubernetes with fixtures, set `KUBERNETES_ENABLED=true` and `KUBERNETES_DEMO_MODE=true`.

Deployment

- **Backend image:** non-root, migrations then serve, health check.
- **Frontend image:** `npm ci`, build-time `NEXT_PUBLIC_*` arguments.
- **Compose:** Postgres, backend and frontend with an explicit CORS origin.
- **CI:** backend lint and tests, frontend lint and production build.

Known issues

Missing favicon	A `/favicon.ico` 404 appears in the console. Cosmetic; no icon asset ships.
Transient RSC 404	During scripted navigation a prefetch of `/login` occasionally logs a 404. Direct loads and all probes return 200; rendering is unaffected.
Live cluster unverified	No Docker daemon or kubeconfig in the verification environment, so only the fixture client was exercised.
Compose unexecuted	The Postgres/Compose path is written but was not run here.
Preview surfaces	Seven pages are intentionally non-functional; they are labelled as previews.

Next milestone

Milestone 3 — investigation engine. Persist investigations, citations and replayable evidence; add a provider abstraction with a fully local mock; then attach the GitHub, Prometheus and Terraform adapters. Authorization and account lifecycle should land before any shared deployment.

Report generated from repository commit `3fec12f` on branch `master`. Screenshots show a locally created demo account and the labelled fixture cluster. No customer, production or live-cluster usage is represented.